

MANUAL DE COMANDOS Y FLUJOS DE TRABAJO EN GIT

GUÍA METODOLÓGICA PARA LA ADMINISTRACIÓN DE RAMAS,
RESOLUCIÓN DE CONFLICTOS, DEPURACIÓN DE ARCHIVOS Y
PUBLICACIÓN EN GITHUB

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

June 1, 2026

RESUMEN

Este documento técnico recopila de forma estructurada los comandos y flujos de trabajo de Git utilizados para administrar el desarrollo de firmware y sistemas de telemetría. Abarca desde la clonación inicial y creación de ramas secundarias de desarrollo (*feature branches*), hasta la resolución de conflictos derivados de archivos de compilación no deseados, la fusión segura de código (*merge*) a la rama principal y la migración selectiva de archivos. Se presentan las mejores prácticas de flujo de trabajo comparando métodos de ramificación e incluye explicaciones detalladas de la sintaxis y comportamiento interno de comandos críticos como **git checkout**, **git rm --cached**, **git reset --hard** y **git remote set-url**.

TABLA DE CONTENIDOS

1	Clonación Local de un Repositorio	4
1.1	Procedimiento detallado	4
2	Gestión de Ramas Secundarias (Feature Branches)	5
2.1	Comandos para la creación e inicio de ramas	5
2.2	Análisis comparativo de flujos de trabajo	5
2.2.1	Método A: Creación anticipada de la rama (Recomendado)	5
2.2.2	Método B: Desarrollo local y ramificación posterior	5
2.3	Confirmación y publicación de la nueva rama	5
3	Cambio de Ramas y Resolución de Bloqueos de Compilación	7
3.1	El problema de archivos de compilación rastreados	7
3.2	La Solución: Limpieza forzada (<code>git reset --hard</code>)	7
4	Eliminación de Archivos del Historial y el uso de <code>.gitignore</code>	8
4.1	El comando de remoción del índice (<code>git rm --cached</code>)	8
4.2	Flujo completo de limpieza permanente	8
5	Integración de Cambios: Fusión (Merge) a la Rama Principal	9
5.1	Flujo de comandos estándar para el Merge	9
5.2	El concepto de fusión automática y resolución de conflictos	9
6	Extracción Selectiva de Archivos entre Ramas (Operaciones Quirúrgicas)	10
6.1	Ejemplo de uso práctico	10
7	Actualización de la Dirección del Servidor Remoto (Remote URL)	11
8	Publicación de un Directorio Local en un Nuevo Repositorio	12
8.1	¿Es necesario crearlo desde la web de GitHub?	12
8.2	¿Qué es GitHub CLI y cómo verificar su instalación?	12
8.3	Método 1: Creación del Repositorio desde la Web de GitHub (Tradicional)	12
8.4	Método 2: Creación y Publicación usando GitHub CLI (<code>gh</code>)	13
8.4.1	¿Qué es <code>gh auth login</code> y cómo autenticarse?	13
9	Eliminación de un Repositorio de GitHub	16
9.1	Método 1: Eliminación desde la Aplicación Web de GitHub	16
9.2	Método 2: Eliminación usando la Consola (GitHub CLI)	16
9.2.1	Ejemplo práctico para eliminar 'LatexRepository':	16
9.3	Resolución del Error de Permisos (HTTP 403: <code>delete_repo scope</code>)	17
9.3.1	Cómo resolverlo:	17
10	Comparación de Diferencias entre Ramas	18
10.1	Comparación desde la Línea de Comandos (Terminal)	18
10.2	Comparación Visual a través del Navegador Web (GitHub)	18
10.2.1	Método 1: Acceso Directo por URL	19
10.2.2	Método 2: Navegando en la Interfaz Gráfica de GitHub	19
10.2.3	Elementos del panel de comparación de GitHub:	19
11	Reubicación de Archivos y Carpetas en el Repositorio	20
11.1	Método 1: Cortar y Pegar en el Explorador de Archivos (Manual)	20
11.2	Método 2: Uso del Comando Nativo de Git (<code>git mv</code>)	20
11.2.1	Ejemplo práctico	20
11.2.2	Confirmar y subir la reubicación	21
12	Actualización de una Rama Secundaria desde la Rama Principal (Main)	22
12.1	¿Por qué es común y necesario?	22
12.2	Flujo de Trabajo Inicial	22
12.3	Método 1: Fusión Directa (<code>git merge</code>) - Seguro y Tradicional	22
12.4	Método 2: Reorganización de la Rama (<code>git rebase</code>) - Historial Lineal	23
12.5	Paso Crítico Posterior: Subir los Cambios a GitHub	23

1 CLONACIÓN LOCAL DE UN REPOSITORIO

El punto de partida de la interacción con el control de versiones es la descarga del repositorio remoto desde GitHub. Este proceso crea una base de datos local en tu disco con el historial completo de commits del proyecto.

1.1 Procedimiento detallado

1. **Obtener la dirección de clonación:** En la página web de tu repositorio en GitHub (por ejemplo, PowerMonitorDashboard), haz clic en el botón verde **Code**, selecciona la pestaña **HTTPS** y copia la URL.
2. **Ejecutar el comando de clonación:** Abre una terminal en tu carpeta de desarrollo local y escribe:

```
git clone https://github.com/rodrigocm1995/PowerMonitorDashboard.git
```

3. **Ingresar al directorio:** Accede a la nueva carpeta creada automáticamente por Git:

```
cd PowerMonitorDashboard
```

2 GESTIÓN DE RAMAS SECUNDARIAS (FEATURE BRANCHES)

Las ramas secundarias se utilizan para implementar nuevas funciones, configurar bases de datos o corregir errores de forma aislada del código de producción estable de la rama **main**.

2.1 Comandos para la creación e inicio de ramas

Para crear una rama (por ejemplo, `feature/add-database`) y cambiarte a ella en una sola instrucción, puedes utilizar la terminal:

```
# Usando el comando clásico:
git checkout -b feature/add-database

# Usando el comando moderno de Git:
git switch -c feature/add-database
```

2.2 Análisis comparativo de flujos de trabajo

Existen dos formas populares para abordar las modificaciones y asignarlas a una nueva rama:

2.2.1 Método A: Creación anticipada de la rama (Recomendado)

Consiste en crear la rama secundaria **antes** de modificar cualquier archivo.

Paso 1: Checkout a la nueva rama

Paso 2: Modificar archivos

Paso 3: Commit y Push

- **Ventajas:** Es el método más limpio y ordenado. Previene por completo la mezcla accidental de cambios de desarrollo con la rama principal en caso de tener que cambiar de contexto de forma imprevista. El historial de commits refleja el inicio exacto de la característica.

2.2.2 Método B: Desarrollo local y ramificación posterior

Consiste en realizar las modificaciones en tu rama actual (ej. **main**) sin hacer commit, y después crear la rama secundaria para confirmar las modificaciones en ella.

Paso 1: Modificar archivos en main

Paso 2: Checkout a nueva rama

Paso 3: Commit y Push

- **Cómo funciona:** En Git, las modificaciones locales que están pendientes de preparación o confirmación (cambios en el *Working Directory*) no pertenecen a ninguna rama. Al cambiarte de rama usando `git checkout -b`, Git arrastra esos archivos modificados contigo de forma automática hacia la nueva rama.
- **Riesgo:** Si el archivo ya tiene cambios diferentes en la rama de destino, Git bloqueará la conmutación arrojando un error de conflicto. Solo es seguro si tu árbol de trabajo no tiene conflictos de fusión directa.

2.3 Confirmación y publicación de la nueva rama

Una vez que has editado y guardado tus archivos, la secuencia de publicación en GitHub es la siguiente:

```
# 1. Verificar el estado del espacio de trabajo
git status

# 2. Agregar los archivos modificados al Staging Area
git add .
```

```
# 3. Confirmar los cambios localmente
git commit -m "Configure database models and context for power monitoring"

# 4. Publicar la rama en GitHub por primera vez
git push -u origin feature/add-database
```

El parámetro `-u` (`--set-upstream`) establece un vínculo de seguimiento de forma que en los próximos envíos bastará con ejecutar simplemente `git push`.

3 CAMBIO DE RAMAS Y RESOLUCIÓN DE BLOQUEOS DE COMPILACIÓN

Para navegar de regreso a una rama existente (como main), se utiliza:

```
git checkout main
# 0 bien:
git switch main
```

3.1 El problema de archivos de compilación rastreados

Un error muy común ocurre al intentar cambiar de rama cuando Git reporta que ciertos archivos se sobrescribirán:

```
1 error: Your local changes to the following files would be overwritten by checkout:
2     Service/TemperatureService/bin/Debug/net9.0/TemperatureService.dll
3     Service/TemperatureService/obj/Debug/net9.0/TemperatureService.dll
4 Please commit your changes or stash them before you switch branches.
5 Aborting
```

¿Por qué sucede? Esto ocurre cuando una de las ramas (como main) tiene en su historial de commits archivos binarios temporales de compilación (como la carpeta bin u obj), y la otra rama no los tiene o los tiene ignorados. Al correr el proyecto (dotnet run o npm start), el compilador re-escribe estos binarios. Al intentar cambiar de rama, Git nota que los archivos rastreados cambiaron localmente y bloquea el proceso para no perder datos.

3.2 La Solución: Limpieza forzada (git reset --hard)

Dado que los archivos reportados son binarios autogenerados de compilación y no contienen código fuente escrito por ti, puedes descartar todas las modificaciones locales de forma segura con el comando:

```
git reset --hard
```

Efecto: Este comando descarta todos los cambios del directorio de trabajo y del área de preparación, regresando el repositorio al estado exacto del último commit registrado. Esto limpia el bloqueo y te permite hacer git switch o git checkout de inmediato.

⊗ Peligro del uso de git reset --hard

El comando git reset --hard destruye permanentemente cualquier cambio que no haya sido confirmado en un commit. Asegúrate de usarlo únicamente si estás seguro de que los archivos listados en rojo por git status son archivos basura de compilación y no código real que desees conservar.

4 ELIMINACIÓN DE ARCHIVOS DEL HISTORIAL Y EL USO DE .GITIGNORE

El archivo `.gitignore` le indica a Git qué archivos debe ignorar por completo (como `node_modules/`, `bin/`, `obj/`). Sin embargo, **el archivo `.gitignore` solo funciona para archivos que no están siendo rastreados (untracked)**.

Si un archivo ya fue subido a GitHub en commits anteriores, Git lo seguirá rastreando e informando sobre sus modificaciones, ignorando las reglas del archivo `.gitignore`.

4.1 El comando de remoción del índice (`git rm --cached`)

Para indicarle a Git que deje de vigilar un archivo o carpeta sin eliminarlo físicamente de tu computadora, debes sacarlo del índice (área de preparación) de Git.

La sintaxis del comando es la siguiente:

```
# Dejar de rastrear una carpeta de compilación de forma recursiva
git rm -r --cached Service/TemperatureService/bin/
git rm -r --cached Service/TemperatureService/obj/
```

git rm: Comando para remover archivos o directorios del control de Git.

-r: Aplica la remoción de manera recursiva (necesaria para procesar carpetas y todo su contenido).

--cached: Modificador crítico. Indica que solo se borre la referencia del archivo de la base de datos de Git (lo que provocará su eliminación del repositorio en la nube de GitHub), pero **mantendrá el archivo intacto en tu almacenamiento local** para que el proyecto siga compilando.

4.2 Flujo completo de limpieza permanente

Para limpiar el historial de la rama actual y hacer que el `.gitignore` empiece a funcionar:

```
# 1. Descartar cambios de compilación locales
git reset --hard

# 2. Remover las carpetas del rastreo de Git
git rm -r --cached Service/TemperatureService/bin/
git rm -r --cached Service/TemperatureService/obj/

# 3. Guardar el cambio y subirlo
git commit -m "Stop tracking build folders"
git push origin <nombre-rama>
```

5 INTEGRACIÓN DE CAMBIOS: FUSIÓN (MERGE) A LA RAMA PRINCIPAL

Cuando una característica desarrollada en una rama secundaria está terminada y validada, se debe fusionar e integrar en la rama principal main.

5.1 Flujo de comandos estándar para el Merge

Para integrar los cambios de la rama feature/add-database dentro de main:

```
# 1. Cambiarse a la rama de destino (main)
git checkout main

# 2. Descargar los últimos cambios del servidor remoto
git pull origin main

# 3. Fusionar la rama secundaria dentro de main
git merge feature/add-database

# 4. Subir la rama main actualizada a GitHub
git push origin main
```

5.2 El concepto de fusión automática y resolución de conflictos

Git analiza el ancestro común más cercano de ambas ramas.

- **Fusión Automática (Fast-forward o 3-way merge):** Si las modificaciones de la rama secundaria se realizaron en archivos o líneas de código diferentes a las que se editaron en main desde la separación, Git fusiona ambos trabajos automáticamente sin alterar nada.
- **Conflictos de fusión:** Si se editó exactamente la misma línea del mismo archivo en ambas ramas con contenidos distintos, Git detendrá la fusión y marcará el archivo con marcas de conflicto para que el desarrollador elija manualmente qué versión conservar antes de finalizar el commit.

6 EXTRACCIÓN SELECTIVA DE ARCHIVOS ENTRE RAMAS (OPERACIONES QUIRÚRGICAS)

En ocasiones no se desea realizar una fusión completa de ramas, sino extraer únicamente un archivo o una carpeta específica de otra rama (por ejemplo, traer el archivo `.gitignore` o la carpeta del microcontrolador `NUCLE0-F303_TempAndLuxMonitor` de la rama `main` hacia tu rama actual).

Para copiar un recurso directamente de una rama a otra sin afectar tus archivos locales editados, se utiliza la siguiente instrucción:

```
git checkout <rama-origen> -- <ruta/del/archivo-o-carpeta>
```

6.1 Ejemplo de uso práctico

Si te encuentras en tu rama de desarrollo y deseas clonar el archivo `.gitignore` tal como está en `main`:

```
git checkout main -- .gitignore
```

- **Comportamiento:** Este comando busca en la base de datos de Git de la rama `main`, toma una copia exacta del archivo `.gitignore` y lo escribe en tu carpeta local activa. Además, lo coloca automáticamente en el área de preparación (*staged*), listo para ser confirmado con un `git commit`.

7 ACTUALIZACIÓN DE LA DIRECCIÓN DEL SERVIDOR REMOTO (REMOTE URL)

Si el nombre del repositorio se modifica en GitHub (por ejemplo, corregir una falta de ortografía o un carácter especial prohibido), se romperá el vínculo local del repositorio. Para corregir la dirección URL de GitHub en tu proyecto local sin perder tu historial, se utiliza el comando:

```
git remote set-url origin https://github.com/rodrigocm1995/AmbientLight-Temperature-Dashboard.git
```

- **Sintaxis:**

- `set-url`: Modifica la dirección URL asociada a un alias de servidor remoto.
- `origin`: Nombre del alias del servidor que se está actualizando.
- La nueva URL `.git` apunta directamente al repositorio renombrado.

Para verificar que el cambio de dirección se aplicó con éxito, ejecuta:

```
git remote -v
```

8 PUBLICACIÓN DE UN DIRECTORIO LOCAL EN UN NUEVO REPOSITORIO

Cuando tienes una carpeta local con archivos (como tu carpeta Latex en C:\Users\casti\OneDrive\Documentos\Latex) y deseas convertirla en un repositorio de GitHub llamado LatexRepository, existen dos caminos para lograrlo: el método tradicional a través de la interfaz web de GitHub y el método avanzado utilizando la línea de comandos.

8.1 ¿Es necesario crearlo desde la web de GitHub?

No es estrictamente necesario entrar al sitio web de GitHub. Puedes crear el repositorio remoto directamente desde tu terminal si tienes instalada la herramienta oficial de línea de comandos de GitHub conocida como **GitHub CLI** (gh). Si no dispones de esta herramienta, sí deberás crear el repositorio vacío manualmente en la página web antes de poder subir tus archivos locales.

8.2 ¿Qué es GitHub CLI y cómo verificar su instalación?

GitHub CLI (representado por el comando de consola gh) es la herramienta oficial de línea de comandos de GitHub. A diferencia de Git (que gestiona el historial local de archivos), GitHub CLI permite realizar tareas directamente sobre la plataforma web de GitHub (como administrar repositorios, solicitudes de extracción o *pull requests*, fallos o *issues*, y secretos de despliegue) sin salir de tu terminal.

Para verificar si ya lo tienes instalado en tu sistema, abre una terminal (PowerShell o CMD) y ejecuta el siguiente comando:

```
gh --version
```

- **Si está instalado:** Verás información detallada con el número de versión (por ejemplo, gh version 2.45.0).
- **Si no está instalado:** Windows arrojará un error indicando que el término gh no se reconoce como un cmdlet, función, archivo de script o programa ejecutable. En este caso, puedes descargarlo e instalarlo desde su sitio oficial: cli.github.com.

8.3 Método 1: Creación del Repositorio desde la Web de GitHub (Tradicional)

Este es el método estándar. Sigue estos pasos para subir tu carpeta local Latex:

1. Crear el repositorio en GitHub:

- a) Inicia sesión en github.com.
- b) Haz clic en el botón **New** (Nuevo) en la sección de repositorios.
- c) Asigna el nombre LatexRepository en **Repository name**.
- d) **Muy Importante:** No selecciones ninguna de las opciones de inicialización (como *Add a README file*, *Add .gitignore* o *Choose a license*). El repositorio remoto debe permanecer completamente vacío para evitar conflictos de fusión iniciales.
- e) Haz clic en el botón **Create repository**.
- f) Copia la URL HTTPS que te proporciona la página (será algo como <https://github.com/rodrigocm1995/LatexRepository.git>).

2. Inicializar y subir la carpeta local desde la terminal: Abre tu consola (PowerShell o CMD) en la ruta de tu carpeta y ejecuta la siguiente secuencia de comandos:

```
# 1. Cambiarse al directorio que deseas subir
cd "C:\Users\casti\OneDrive\Documentos\Latex"

# 2. Inicializar el repositorio Git local en esta carpeta
```

```
git init

# 3. Agregar todos los archivos al area de preparacion
git add .

# 4. Registrar los archivos en tu primer commit local
git commit -m "Initial commit: Uploading LaTeX documents"

# 5. Asegurar que la rama principal se llame main
git branch -M main

# 6. Vincular el repositorio local con el remoto de GitHub
git remote add origin https://github.com/rodrigocm1995/LatexRepository.git

# 7. Subir el historial de commits a la rama main
git push -u origin main
```

8.4 Método 2: Creación y Publicación usando GitHub CLI (gh)

Si tienes instalado **GitHub CLI**, puedes crear el repositorio en la nube y subir tu código en un solo paso directo sin abrir el navegador, siempre y cuando tu terminal esté autenticada con tu cuenta de GitHub.

8.4.1 ¿Qué es `gh auth login` y cómo autenticarse?

El comando `gh auth login` es el mecanismo que utiliza GitHub CLI para vincular tu terminal local con tu cuenta en la nube de GitHub, otorgándole los permisos necesarios para realizar cambios remotos (como crear repositorios o subir código).

1. **Cómo saber si ya estás autenticado:** Ejecuta el siguiente comando en tu terminal para verificar el estado de tu sesión activa:

```
gh auth status
```

- **Sesión activa:** La consola te responderá con tu nombre de usuario e información de conexión (por ejemplo: Logged in to github.com account rodrigocm1995).
- **Sin sesión:** La consola te indicará que no has iniciado sesión en ningún host de GitHub.

2. **Cómo realizar la autenticación:** Si no has iniciado sesión, ejecuta:

```
gh auth login
```

A continuación, la terminal iniciará un asistente interactivo donde debes responder las siguientes opciones (figura 1):

```
C:\Users\casti>gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 0317-A2F1
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as rodrigocm1995
```

Figura 1: Opciones del asistente interactivo de `gh auth login`.

- a) **What account do you want to log into?** Selecciona **GitHub.com**.
- b) **What is your preferred protocol for Git operations?** Selecciona **HTTPS**.
- c) **Authenticate Git with your GitHub credentials?** Selecciona **Yes** (esto configura automáticamente las credenciales para futuros comandos de Git).

- d) **How would you like to authenticate GitHub CLI?** Selecciona Login with a web browser.
- e) La terminal generará un código único de 8 caracteres (por ejemplo: A1B2-C3D4) y te solicitará presionar **Enter** para abrir tu navegador web predeterminado.
- f) En la página de GitHub que se abrirá, introduce el código de 8 caracteres (figura 2).

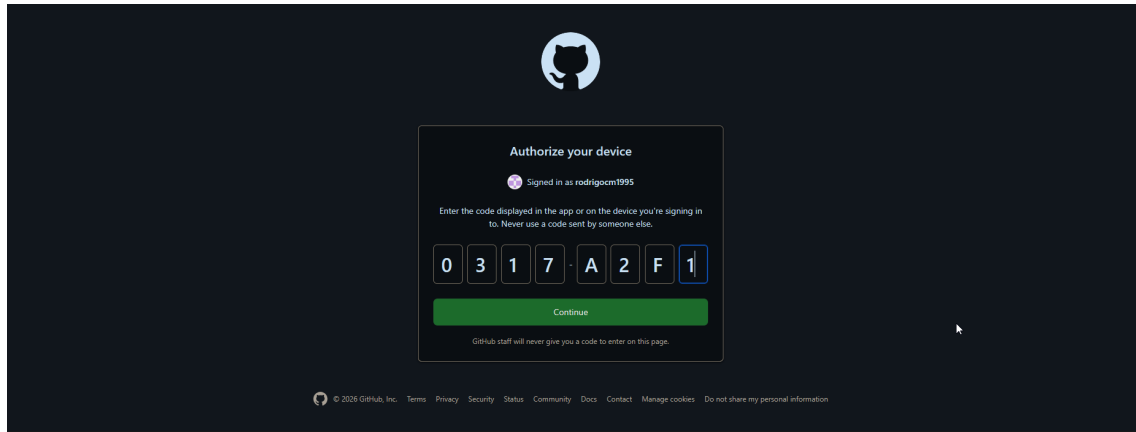


Figura 2: Ingreso del código de autorización.

- g) Haz clic en **Authorize github**. La terminal confirmará el inicio de sesión exitoso de inmediato (figura 3).

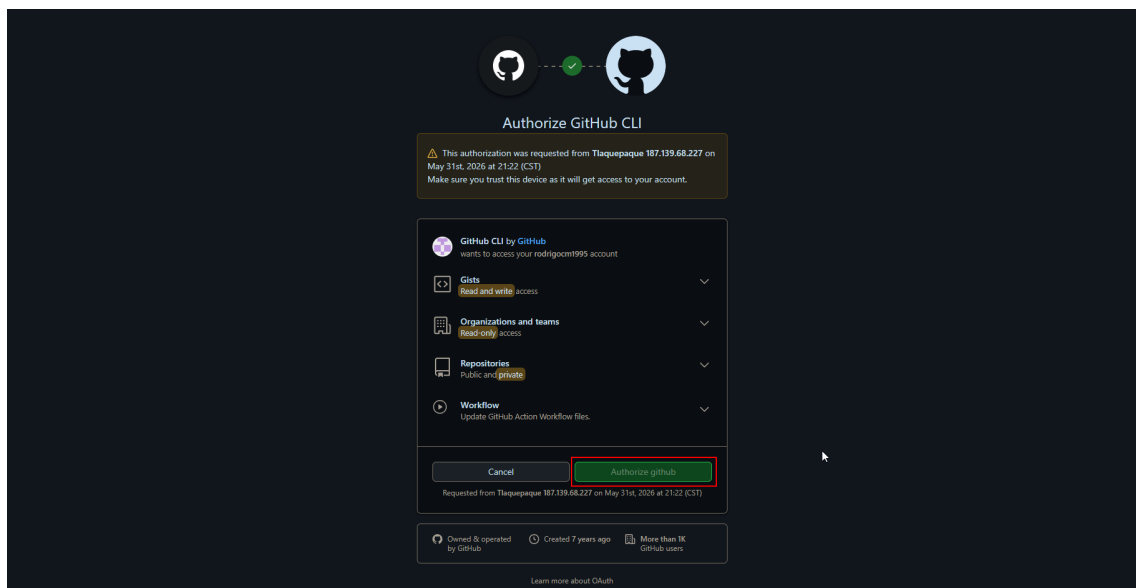


Figura 3: Autorizar que GitHub CLI acceda a la cuenta.

Una vez autenticado, abre la terminal en la carpeta correspondiente y ejecuta:

```
# 1. Acceder a la carpeta local
cd "C:\Users\casti\OneDrive\Documentos\Latex"

# 2. Inicializar el repositorio local si no lo has hecho
git init

# 3. Preparar y confirmar los archivos localmente
git add .
git commit -m "Initial commit: Uploading LaTeX documents"

# 4. Crear el repositorio en GitHub y hacer Push automáticamente
gh repo create LatexRepository --public --source=. --remote=origin --push
```

Explicación de parámetros de GitHub CLI

- public:** Crea el repositorio en GitHub con visibilidad pública (usa **--private** si deseas que sea privado).
- source=.**: Indica que el código fuente base se encuentra en el directorio actual (.).
- remote=origin:** Configura automáticamente el enlace remoto con el nombre origin.
- push:** Realiza la subida de los commits confirmados en tu rama principal inmediatamente después de crear el repositorio remoto.

9 ELIMINACIÓN DE UN REPOSITORIO DE GITHUB

En caso de que requieras eliminar de forma permanente un repositorio de GitHub (por ejemplo, el repositorio de prueba `LatexRepository`), existen dos maneras de llevarlo a cabo: a través de la interfaz web tradicional o utilizando la consola de comandos mediante GitHub CLI.

⚠ Advertencia sobre la eliminación de repositorios

La eliminación de un repositorio en GitHub es una acción destructiva e irreversible. Al hacerlo, se perderá de forma permanente todo el historial de commits, ramas, issues, pull requests y comentarios asociados. Asegúrate de respaldar tu código local antes de proceder.

9.1 Método 1: Eliminación desde la Aplicación Web de GitHub

Este es el proceso estándar a través de tu navegador de internet:

1. Accede a tu cuenta de GitHub y navega a la página principal del repositorio que deseas borrar (ej. `rodrigocm1995/LatexRepository`).
2. Haz clic en la pestaña **Settings** (Configuración) ubicada en el menú horizontal superior del repositorio (representada por un icono de engranaje).
3. Desplázate hasta el final de la página de configuración hasta encontrar la sección denominada **Danger Zone** (Zona de Peligro).
4. Identifica la opción **Delete this repository** (Eliminar este repositorio) y haz clic sobre ella.
5. GitHub te mostrará una serie de advertencias sobre lo que implica la eliminación. Haz clic en los botones de confirmación correspondientes.
6. Para evitar eliminaciones accidentales, la plataforma te solicitará escribir de forma explícita la ruta completa del repositorio (por ejemplo, `rodrigocm1995/LatexRepository`).
7. Finalmente, haz clic en el botón de confirmación definitiva. Es posible que debas ingresar tu contraseña de GitHub o código de doble factor (2FA) para finalizar el proceso.

9.2 Método 2: Eliminación usando la Consola (GitHub CLI)

Si tienes instalada y configurada la herramienta de comandos oficial de GitHub (`gh`), puedes eliminar el repositorio remoto directamente desde tu terminal (PowerShell o CMD) sin abrir el navegador.

Para ello, utiliza el comando `gh repo delete` con la siguiente sintaxis:

```
gh repo delete <propietario>/<nombre-repositorio>
```

9.2.1 Ejemplo práctico para eliminar 'LatexRepository':

Abre tu terminal y ejecuta:

```
gh repo delete rodrigocm1995/LatexRepository
```

- **Comportamiento por defecto:** El comando te pedirá confirmación de forma interactiva en la terminal escribiendo de nuevo el nombre del repositorio para confirmar.
- **Eliminación directa sin confirmación interactiva:** Si deseas saltarte la confirmación manual (por ejemplo, en scripts automatizados), puedes añadir el modificador `--yes`:

```
gh repo delete rodrigocm1995/LatexRepository --yes
```

9.3 Resolución del Error de Permisos (HTTP 403: delete_repo scope)

Por razones de seguridad, cuando inicias sesión por primera vez en GitHub CLI con `gh auth login`, la herramienta solicita permisos limitados que no incluyen la capacidad de borrar repositorios. Si intentas borrar un repositorio sin este alcance, obtendrás el siguiente error:

```
1 HTTP 403: Must have admin rights to Repository.  
2 This API operation needs the "delete_repo" scope. To request it, run:  
3 gh auth refresh -h github.com -s delete_repo
```

9.3.1 Cómo resolverlo:

Para otorgarle el permiso de eliminación a GitHub CLI, debes actualizar las credenciales de tu cuenta ejecutando el siguiente comando:

```
gh auth refresh -h github.com -s delete_repo
```

1. El comando te solicitará abrir el navegador web para re-autorizar GitHub CLI. Presiona la tecla `Enter` para proceder.
2. En el navegador, se abrirá la página de GitHub donde aparecerá la solicitud del permiso adicional: **Delete repositories** (Eliminar repositorios). Haz clic en el botón verde **Authorize github**.
3. Tras la confirmación, regresa a tu terminal y vuelve a ejecutar el comando de eliminación original. Ahora se completará con éxito sin errores de privilegios.

10 COMPARACIÓN DE DIFERENCIAS ENTRE RAMAS

Para proyectos colaborativos o desarrollos extensos (por ejemplo, comparar la rama `feature/sensor-calibration` con `feature/add-database` en el proyecto `PowerMonitorDashboard`), Git y GitHub ofrecen herramientas sumamente potentes para contrastar qué archivos o carpetas se han agregado, modificado o eliminado entre ambas ramas.

10.1 Comparación desde la Línea de Comandos (Terminal)

La herramienta principal para contrastar dos ramas es el comando `git diff`. Dependiendo del nivel de detalle que requieras, dispones de los siguientes modificadores:

1. **Ver el estado de los archivos (Recomendado):** Si deseas una lista limpia de los archivos que difieren sin saturar la terminal con líneas de código, utiliza el parámetro `--name-status`:

```
git diff --name-status feature/sensor-calibration..feature/add-database
```

Interpretación del resultado:

- A (Added): El archivo fue **Agregado** en la rama comparada (existe en `feature/add-database` pero no en la rama base).
 - M (Modified): El archivo fue **Modificado** entre ambas ramas.
 - D (Deleted): El archivo fue **Eliminado** en la rama comparada (o solo existía en la rama base).
2. **Ver un resumen estadístico del código cambiado:** Si deseas saber cuántas líneas se agregaron o eliminaron en cada archivo afectado, utiliza `--stat`:

```
git diff --stat feature/sensor-calibration..feature/add-database
```

Esto generará una salida informativa que indica el total de cambios cuantitativos:

```
1 Service/PowerMonitorService/Program.cs | 15 ++++++-----
2 View/PowerMonitorView/src/app/app.component.html | 34 ++++++
3 2 files changed, 45 insertions(+), 4 deletions(-)
4
```

3. **Obtener solo la ruta de los archivos modificados:** Si requieres únicamente la lista de nombres y directorios de los archivos para copiar y pegar, usa `--name-only`:

```
git diff --name-only feature/sensor-calibration..feature/add-database
```

4. **Ver la diferencia exacta de código línea por línea:** Para revisar el código modificado directamente desde la consola, ejecuta el comando sin parámetros extra:

```
git diff feature/sensor-calibration..feature/add-database
```

Significado de la sintaxis de dos puntos (..)

La sintaxis `ramaA..ramaB` le indica a Git que realice la comparación de `ramaB` utilizando como base de referencia a `ramaA`. Por lo tanto, el resultado te mostrará qué cambios (adiciones, ediciones o eliminaciones) deben realizarse para transformar la rama base en la rama comparada.

10.2 Comparación Visual a través del Navegador Web (GitHub)

Si prefieres un entorno más gráfico e interactivo que facilite la navegación por los directorios de cambios, la plataforma web de GitHub cuenta con una herramienta excelente.

10.2.1 Método 1: Acceso Directo por URL

La forma más veloz de ingresar a la comparación visual es estructurando la dirección en la barra de navegación de tu navegador de la siguiente manera:

```
1 https://github.com/<usuario>/<repositorio>/compare/<rama_base>...<rama_a_comparar>
```

En tu caso particular, la dirección exacta para comparar las ramas de tu proyecto de telemetría de energía es:

```
1 https://github.com/rodrigocm1995/PowerMonitorDashboard/compare/feature/sensor-calibration
...feature/add-database
```

10.2.2 Método 2: Navegando en la Interfaz Gráfica de GitHub

1. Abre tu repositorio en github.com.
2. Haz clic sobre el botón **Branches** (Ramas) ubicado en la parte superior izquierda (el cual indica el número de ramas activas de tu proyecto).
3. Localiza la rama que deseas contrastar (por ejemplo, feature/add-database) y haz clic en el botón **Compare** (Comparar) o **New pull request** a su derecha.
4. En la parte superior de la pantalla de comparación, selecciona los dos desplegables:
 - **base:** La rama de referencia (ej. feature/sensor-calibration).
 - **compare:** La rama con las modificaciones (ej. feature/add-database).

10.2.3 Elementos del panel de comparación de GitHub:

Una vez cargada la pantalla, la herramienta visual te proporcionará dos pestañas:

- **Commits:** Una lista ordenada con todos los mensajes de confirmación de cambios presentes en la rama comparada.
- **Files changed (Archivos modificados):** Muestra el árbol de directorios a la izquierda para navegar entre carpetas, y el código coloreado a la derecha (verde para adiciones, rojo para eliminaciones). Los archivos nuevos se identifican con una etiqueta verde de letra **A** (Added).

11 REUBICACIÓN DE ARCHIVOS Y CARPETAS EN EL REPOSITORIO

Cuando es necesario mover archivos o carpetas dentro del repositorio (por ejemplo, mover la carpeta del microcontrolador NUCLE0-F303_TMP117 de la ruta TMP117/TMP117 a TMP117/Examples), existen dos formas principales para realizar esta acción de forma segura en Git sin perder el historial de confirmaciones (*commit history*).

11.1 Método 1: Cortar y Pegar en el Explorador de Archivos (Manual)

Este método consiste en utilizar el entorno gráfico de tu sistema operativo (como el Explorador de Archivos de Windows o VS Code) para mover los archivos, y posteriormente indicarle a Git que procese dichos cambios.

1. **Mover los archivos:** Abre el Explorador de archivos y realiza la operación tradicional de cortar (**Ctrl** + **X**) y pegar (**Ctrl** + **V**) para reubicar la carpeta o archivos en su nueva ruta de destino.
2. **Preparar los cambios en Git:** Abre una terminal en la raíz del repositorio y ejecuta:

```
git add .
```

- **Comportamiento interno:** Git no guarda de forma explícita el concepto de “renombrar” o “mover”. En su lugar, al realizar el `git add .`, Git detecta que los archivos desaparecieron de la ruta anterior y aparecieron en la nueva ruta con el mismo contenido (o casi idéntico).

3. **Confirmar y publicar los cambios:**

```
git commit -m "Mover NUCLE0-F303_TMP117 a la carpeta de Examples"
git push origin <nombre-rama>
```

Al hacer el commit, el algoritmo de detección de renombrado de Git (*rename detection*) marcará automáticamente la reubicación como un cambio tipo rename (100% match), conservando todo su historial en los nuevos commits.

11.2 Método 2: Uso del Comando Nativo de Git (`git mv`)

Este método es el más limpio y recomendado cuando se trabaja directamente desde la consola, ya que le indica a Git de forma explícita la reubicación del recurso. Este comando realiza el movimiento físico en el almacenamiento y prepara (*stages*) la reubicación en el índice en una sola instrucción.

La sintaxis del comando es la siguiente:

```
git mv <ruta/actual/del/recurso> <nueva/ruta/de/destino>
```

11.2.1 Ejemplo práctico

Si te encuentras en la raíz del repositorio y deseas mover la carpeta del sensor del microcontrolador a la carpeta de ejemplos utilizando la terminal:

```
git mv C/TMP117/TMP117/NUCLE0-F303_TMP117 C/TMP117/Examples/
```

Rutas en Windows

Al ejecutar comandos de Git en Windows (usando Git Bash, PowerShell o CMD), se recomienda utilizar barras diagonales hacia adelante (/) para evitar que la terminal interprete la barra invertida (\) como un carácter de escape o genere errores de resolución de ruta.

11.2.2 *Confirmar y subir la reubicación*

Dado que `git mv` ya preparó la reubicación en el área de preparación, no es necesario hacer un `git add`. Simplemente ejecuta:

```
git commit -m "Mover NUCLE0-F303_TMP117 a la carpeta de Examples"  
git push origin <nombre-rama>
```

12 ACTUALIZACIÓN DE UNA RAMA SECUNDARIA DESDE LA RAMA PRINCIPAL (MAIN)

En el desarrollo de software colaborativo o en proyectos individuales con varias ramas de características (*feature branches*), es una **práctica sumamente común y recomendada** integrar periódicamente los cambios más recientes de la rama principal (main) hacia tu rama secundaria de trabajo.

12.1 ¿Por qué es común y necesario?

A medida que pasa el tiempo, la rama main puede avanzar debido a que se fusionan otras ramas, se corrigen errores (hotfixes) o se actualizan configuraciones y dependencias globales. Mantener tu rama secundaria actualizada con respecto a main te permite:

- **Detectar y resolver conflictos tempranamente** en tu entorno de desarrollo local, en lugar de enfrentarte a conflictos masivos al momento de finalizar tu característica y hacer el merge de retorno.
- **Asegurar la compatibilidad** de tu código nuevo con las modificaciones más recientes hechas por otros miembros del equipo.
- **Disponer de las últimas librerías, dependencias o refactorizaciones** globales para continuar tu desarrollo sin inconsistencias.

12.2 Flujo de Trabajo Inicial

Antes de integrar los cambios, debes asegurar que tu rama main local tiene las últimas modificaciones del servidor remoto de GitHub:

```
# 1. Cambiarse a la rama principal
git checkout main

# 2. Descargar los últimos cambios de GitHub
git pull origin main

# 3. Regresar a tu rama secundaria de trabajo
git checkout feature/nombre-de-tu-rama
```

Una vez realizados estos pasos, puedes elegir uno de los siguientes dos métodos para integrar la rama main en tu rama secundaria:

12.3 Método 1: Fusión Directa (git merge) - Seguro y Tradicional

Este método toma todos los cambios nuevos de main y los inserta en tu rama secundaria a través de un commit especial de fusión (*merge commit*).

```
git merge main
```

- **Comportamiento ante conflictos:** Si existen conflictos de código en las mismas líneas, Git detendrá el merge. Deberás resolver los conflictos manualmente en los archivos correspondientes, prepararlos con `git add` y guardar la fusión realizando un commit.
- **Ventajas:** No reescribe ni altera el historial de tus commits. Es ideal para ramas compartidas con otros desarrolladores.
- **Desventajas:** Genera commits de fusión automáticos (ej. *Merge branch 'main' into...*) que pueden saturar visualmente el historial de ramas en repositorios grandes.

12.4 Método 2: Reorganización de la Rama (`git rebase`) - Historial Lineal

Este método "despega" temporalmente tus commits específicos de la rama secundaria, coloca los commits nuevos de main en la base de la rama, y finalmente vuelve a aplicar tus commits uno por uno encima del código más actualizado.

```
git rebase main
```

- **Comportamiento ante conflictos:** Git pausará la reorganización al encontrar un conflicto. Deberás resolverlo, prepararlo con `git add` y reanudar con `git rebase -continue`.
- **Ventajas:** Elimina commits de fusión innecesarios, manteniendo un historial de confirmaciones lineal, limpio y fácil de leer.
- **Desventajas y Precaución:** Dado que altera el identificador SHA-1 de tus commits locales, si ya habías subido anteriormente la rama a GitHub, deberás forzar la publicación de tus cambios utilizando el parámetro `-force-with-lease`:

```
git push origin feature/nombre-de-tu-rama --force-with-lease
```

12.5 Paso Crítico Posterior: Subir los Cambios a GitHub

Un error sumamente frecuente tras completar con éxito una fusión (`git merge`) o reorganización (`git rebase`) local es asumir que los cambios ya se encuentran en la plataforma web de GitHub.

⚠ Operaciones 100% Locales

Tanto `git merge` como `git rebase` son comandos que actúan exclusivamente sobre la base de datos de Git local en tu computadora. Hasta que no realices una subida explícita, la versión de la rama en GitHub permanecerá en su estado anterior.

Para que tus cambios se actualicen en el servidor de GitHub y se reflejen en la web, debes ejecutar:

```
# Si utilizaste el Metodo 1 (git merge main):
git push origin feature/nombre-de-tu-rama

# Si utilizaste el Metodo 2 (git rebase main):
git push origin feature/nombre-de-tu-rama --force-with-lease
```